9.5 Adaptable Priority Queues

The methods of the priority queue ADT given in Section 9.1.2 are sufficient for most basic applications of priority queues, such as sorting. However, there are situations in which additional methods would be useful, as shown by the scenarios below involving the standby airline passenger application.

- A standby passenger with a pessimistic attitude may become tired of waiting and decide to leave ahead of the boarding time, requesting to be removed from the waiting list. Thus, we would like to remove from the priority queue the entry associated with this passenger. Operation remove_min does not suffice since the passenger leaving does not necessarily have first priority. Instead, we want a new operation, remove, that removes an arbitrary entry.
- Another standby passenger finds her gold frequent-flyer card and shows it to the agent. Thus, her priority has to be modified accordingly. To achieve this change of priority, we would like to have a new operation update allowing us to replace the key of an existing entry with a new key.

We will see another application of adaptable priority queues when implementing certain graph algorithms in Sections 14.6.2 and 14.7.1.

In this section, we develop an ***adaptable priority queue*** ADT and demonstrate how to implement this abstraction as an extension to our heap-based priority queue.

9.5.1 Locators

In order to implement methods update and remove efficiently, we need a mechanism for finding a user's element within a priority queue that avoids performing a linear search through the entire collection. To support our goal, when a new element is added to the priority queue, we return a special object known as a ***locator*** to the caller. We then require the user to provide an appropriate locator as a parameter when invoking the update or remove method, as follows, for a priority queue P:

P.update(loc, k, v): Replace key and value for the item identified by locator loc.

P.remove(loc): Remove the item identified by locator loc from the priority queue and return its (key,value) pair.

The locator abstraction is somewhat akin to the Position abstraction used in our positional list ADT from Section 7.4, and our tree ADT from Chapter 8. However, we differentiate between a locator and a position because a locator for a priority queue does not represent a tangible placement of an element within the structure. In our priority queue, an element may be relocated within our data structure during an operation that does not seem directly relevant to that element. A locator for an item will remain valid, as long as that item remains somewhere in the queue.

9.5.2 Implementing an Adaptable Priority Queue

In this section, we provide a Python implementation of an adaptable priority queue as an extension of our HeapPriorityQueue class from Section 9.3.4. To implement a Locator class, we will extend the existing _Item composite to add an additional field designating the current index of the element within the array-based representation of our heap, as shown in Figure 9.10.

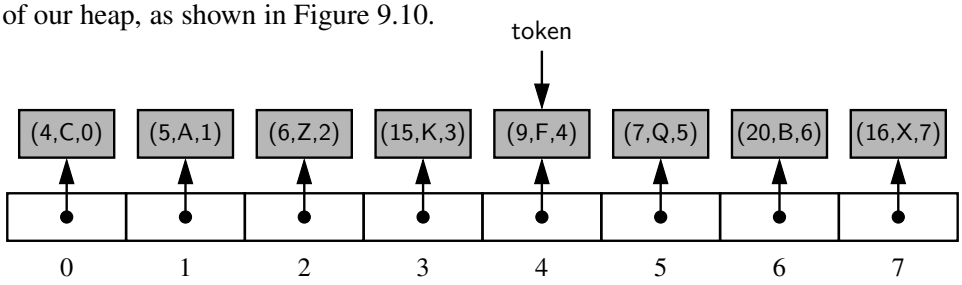


Figure 9.10: Representing a heap using a sequence of locators. The third element of each locator instance corresponds to the index of the item within the array. Identifier token is presumed to be a locator reference in the user's scope.

The list is a sequence of references to locator instances, each of which stores a key, value, and the current index of the item within the list. The user will be given a reference to the Locator instance for each inserted element, as portrayed by the token identifier in Figure 9.10.

When we perform priority queue operations on our heap, and items are relocated within our structure, we reposition the locator instances within the list and we update the third field of each locator to reflect its new index within the list. As an example, Figure 9.11 shows the state of the above heap after a call to remove_min( ). The heap operation caused the minimum entry, (4,C), to be removed, and the entry, (16,X), to be temporarily moved from the last position to the root, followed by a down-heap bubble phase. During the down-heap, element (16,X) was swapped

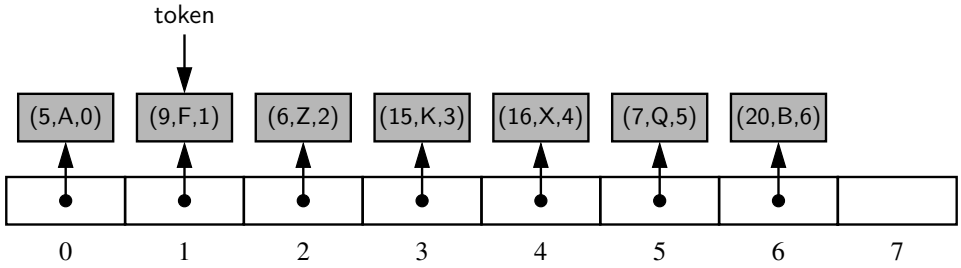


Figure 9.11: The result of a call to remove_min( ) on the heap originally portrayed in Figure 9.10. Identifier token continues to reference the same locator instance as in the original configuration, but the placement of that locator in the list has changed, as has the third field of the locator.

with its left child, (5,A), at index 1 of the list, then swapped with its right child, (9,F), at index 4 of the list. In the final configuration, the locator instances for all affected elements have been modified to reflect their new location.

It is important to emphasize that the locator instances have not changed identity. The user's token reference, portrayed in Figures 9.10 and 9.11, continues to reference the same instance; we have simply changed the third field of that instance, and we have changed where that instance is referenced within the list sequence.

With this new representation, providing the additional support for the adaptable priority queue ADT is rather straightforward. When a locator instance is sent as a parameter to update or remove, we may rely on the third field of that structure to designate where the element resides in the heap. With that knowledge, the update of a key may simply require an up-heap or down-heap bubbling step to reestablish the heap-order property. (The complete binary tree property remains intact.) To implement the removal of an arbitrary element, we move the element at the last position to the vacated location, and again perform an appropriate bubbling step to satisfy the heap-order property.

Python Implementation

Code Fragments 9.8 and 9.9 present a Python implementation of an adaptable priority queue, as a subclass of the HeapPriorityQueue class from Section 9.3.4. Our modifications to the original class are relatively minor. We define a public Locator class that inherits from the nonpublic _Item class and augments it with an additional _index field. We make it a public class because we will be using locators as return values and parameters; however, the public interface for the locator class does not include any other functionality for the user.

To update locators during the flow of our heap operations, we rely on an intentional design decision that our original class uses a nonpublic _swap method for all data movement. We override that utility to execute the additional step of updating the stored indices within the two swapped locator instances.

We provide a new _bubble utility that manages the reinstatement of the heap-order property when a key has changed at an arbitrary position within the heap, either due to a key update, or the blind replacement of a removed element with the item from the last position of the tree. The _bubble utility determines whether to apply up-heap or down-heap bubbling, depending on whether the given location has a parent with a smaller key. (If an updated key coincidentally remains valid for its current location, we technically call _downheap but no swaps result.)

The public methods are provided in Code Fragment 9.9. The existing add method is overridden, both to make use of a Locator instance rather than an _Item instance for storage of the new element, and to return the locator to the caller. The remainder of that method is similar to the original, with the management of locator indices enacted by the use of the new version of _swap. There is no reason to over-

ride the remove_min method because the only change in behavior for the adaptable priority queue is again provided by the overridden _swap method.

The update and remove methods provide the core new functionality for the adaptable priority queue. We perform robust checking of the validity of a locator that is sent by a caller (although in the interest of space, our displayed code does not do preliminary type-checking to ensure that the parameter is indeed a Locator instance). To ensure that a locator is associated with a current element of the given priority queue, we examine the index that is encapsulated within the locator object, and then verify that the entry of the list at that index is the very same locator.

In conclusion, the adaptable priority queue provides the same asymptotic efficiency and space usage as the nonadaptive version, and provides logarithmic performance for the new locator-based update and remove methods. A summary of the performance is given in Table 9.4.

```
1  class AdaptableHeapPriorityQueue(HeapPriorityQueue):
2    """A locator-based priority queue implemented with a binary heap."""
3
4    #------------------------------ nested Locator class ------------------------------
5    class Locator(HeapPriorityQueue._Item):
6      """Token for locating an entry of the priority queue."""
7      __slots__ = '_index'                    # add index as additional field
8
9      def __init__(self, k, v, j):
10       super( ).__init__(k,v)
11       self._index = j
12
13   #------------------------------ nonpublic behaviors ------------------------------
14   # override swap to record new indices
15   def _swap(self, i, j):
16     super( )._swap(i,j)                     # perform the swap
17     self._data[i]._index = i                # reset locator index (post-swap)
18     self._data[j]._index = j                # reset locator index (post-swap)
19
20   def _bubble(self, j):
21     if j > 0 and self._data[j] < self._data[self._parent(j)]:
22       self._upheap(j)
23     else:
24       self._downheap(j)
```

Code Fragment 9.8: An implementation of an adaptable priority queue (continued in Code Fragment 9.9). This extends the HeapPriorityQueue class of Code Fragments 9.4 and 9.5

```
25    def add(self, key, value):
26      """Add a key-value pair."""
27      token = self.Locator(key, value, len(self._data)) # initiaize locator index
28      self._data.append(token)
29      self._upheap(len(self._data) - 1)
30      return token
31
32    def update(self, loc, newkey, newval):
33      """Update the key and value for the entry identified by Locator loc."""
34      j = loc._index
35      if not (0 <= j < len(self) and self._data[j] is loc):
36        raise ValueError('Invalid locator')
37      loc._key = newkey
38      loc._value = newval
39      self._bubble(j)
40
41    def remove(self, loc):
42      """Remove and return the (k,v) pair identified by Locator loc."""
43      j = loc._index
44      if not (0 <= j < len(self) and self._data[j] is loc):
45        raise ValueError('Invalid locator')
46      if j == len(self) - 1:                  # item at last position
47        self._data.pop( )                     # just remove it
48      else:
49        self._swap(j, len(self)-1)            # swap item to the last position
50        self._data.pop( )                     # remove it from the list
51        self._bubble(j)                       # fix item displaced by the swap
52      return (loc._key, loc._value)
```

Code Fragment 9.9: An implementation of an adaptable priority queue (continued from Code Fragment 9.8).

Operation	Running Time
len(P), P.is_empty( ), P.min( )	$O(1)$
P.add(k,v)	$O(\log n)^*$
P.update(loc, k, v)	$O(\log n)$
P.remove(loc)	$O(\log n)^*$
P.remove_min( )	$O(\log n)^*$

*amortized with dynamic array

Table 9.4: Running times of the methods of an adaptable priority queue, P, of size n, realized by means of our array-based heap representation. The space requirement is $O(n)$.